Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251229**
Nama Lengkap	**Ignatius Harya Nugraha**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Definisi Modular Programming

Apa sih yang dimakasud programming?

Konsep dasar dari Modular Programming adalah pemecahan program besar menjadi bagian-bagian kecil yang lebih terorganisir dan mudah dikelola. Bagian-bagian kecil inilah yang disebut fungsi (function).

Dalam dunia nyata seperti halnya saat kita membangun rumah dimana kita tidak langsung membuatnya namun kita memisah setiap tahapan mennjadi beberapa bagian. Bagian bagian tersebut lalu digabung menjadi satu dan dapat digunakan di ruangan lain yang akan dibangun. Dan bagian bagian tersebut disebut sebagai fungsi.

Pada Bahasa pemrograman python kita bisa menggunakan fungsi bawaan yang dapat kita pakai yaitu :

- **Input()** yang memiliki fungsi untuk membaca input pengguna
- **Print()** untuk menampilkan output ke terminal atau layer
- **Len()** yang memiliki fungsi untuk menghitung Panjang suatu data

Selain fungsi tersebut kita juga dapat membuat fungsi yang kita mau yang telah dipelajari pada minggu ke 5 kemarin.

Fungsi

Struktur dasar Fungsi

Untuk membuat fungsi di Python, kita dapat menggunakan perintah def lalu diikuti dengan nama fungsi dan tanda kurung. Fungsi dalam python biasanya ditulis dengan format seperti ini:

```
def namaFungsi(a,b):
    hasil = a + b
    return hasil
    #atau dapat ditulis dengan fungsi yang diinginkan

print(namaFungsi(1,2))
```

dalam kode tersebut bisa diartikan sebagai berikut:

- **def** adalah kata kunci untuk mendefinisikan/membuat fungsi baru.
- **tambah** adalah nama fungsi yang kita buat. Nama ini bebas, tapi sebaiknya mencerminkan kegunaan fungsi.
- **a dan b** adalah parameter, yaitu "kotak penampung" untuk nilai yang kita kirimkan ke fungsi.
- **return** digunakan untuk mengeluarkan nilai hasil dari fungsi agar bisa digunakan di luar fungsi.

Cara Kerja Fungsi

Untuk memahami caraa kerja fungsi akan lebih baik jika dijelaskan menggunakan contoh program maka saya akan menggunakan program tadi sebagai contohnya:

```
def namaFungsi(a,b):
    hasil = a + b
    return hasil
    #atau dapat ditulis dengan fungsi yang diinginkan

print(namaFungsi(1,2))
```

dalam kode tersebut dapat dibedakan menjadi 2 bagian yaitu bagian 1:

```
def namaFungsi(a,b):
    hasil = a + b
    return hasil
```

bagian ini adalah definisi fungsi yang berguna untuk inisiasi fungsi yang di inginkan.

lalu ada bagian dua yang berisi:

```
print(namaFungsi(1,2))
```

Bagian ini memiliki fungsi untuk memanggil fungsi yang telah dibuat dan mengeluarkan nilainnya kepada computer dari hasil fungsi.

Urutan eksekusinya adalah sebagai berikut:

1. Python membaca baris 1-3 (definisi fungsi), tapi belum menjalankannya.
2. Di baris 6, fungsi namaFungsi() dipanggil dengan argumen 1 dan 2. Program melompat ke baris 1.
3. Parameter a diisi 1, dan b diisi 2.
4. Baris 2 dijalankan: hasil = 1 + 2 = 3.
5. Baris 3 dijalankan: return 3. Fungsi selesai dan mengembalikan nilai 15 ke baris 4.
6. Kembali ke baris 4: variabel c diisi dengan 15 lalu python menampilkan hasil tersebut menggunakan dungsi print().

Return Value

Setelah kita belajar fungsi yang dapat mengembalikan nilai, ternyata ada juga jenis fungsi yang tidak mengembalikan nilai atau return value dan itu disebut Void Function. Sehingga karena ada 2 macam fungsi, fungsi dapat dibagi menjadi 2 bagian yaitu :

- **Void Function** dimana fungsi hanya melakukan sesuatu tetapi tidak menghasilkan nilai, atau jika di print akan menghasilkan nilai none
- **Value Function** dimana fungsi ini akan melakukaan sesuatu seperti penambahan atau pemrosesan yang lain lalu mengembalikan nilai atau hasil yang dapat disimpan kedalam variable.

Setelah pembahasan definisi mengenai jenis jenis fungsi berikut saya tunjukkan contoh dari Void Function dan Value Function.

Contoh Void Function

Fungsi berikut hanya menampilkan pesan, tidak mengembalikan nilai apa pun:

```
def sapa(nama):
    print("Halo,", nama, "! Selamat datang.")

sapa("Budi")
hasil = sapa("Budi")
print(hasil)
```

```
[Running] python -u "d:\Project\Kuliah\PrakAlPro-71251229\Minggu_5\fungsi.py"
Halo, Budi ! Selamat datang.
Halo, Budi ! Selamat datang.
None
```

Nah karena kita disini menggunakan void function atau fungsi tanpa pemgembalian nilai, disinikita mendapatkan None Ketika fungsi tersebut di print

Contoh Return Function

Fungsi berikut mengembalikan nilai rata-rata dari tiga bilangan:

```
def tambah(a, b, c):
    hasil = a + b + c
    return hasil

nilai1 = 70
nilai2 = 85
nilai3 = 55

rata_rata = tambah(nilai1, nilai2, nilai3) / 3
print(rata_rata)
```

```
[Running] python -u "d:\Project\Kuliah\PrakAlPro-71251229\Minggu_5\fungsi.py"
70.0
```

Karena ini adalah return function maka Ketika di print akan menghasilkan nilai 70 dan tidak mungkin None.

Pada contoh di atas, return digunakan untuk dua hal sekaligus, yaitu:

- Mengeluarkan nilai hasil dari dalam fungsi ke luar.
- Menghentikan/mengakhiri eksekusi fungsi.

Optional Argument dan Named Argument

Optional Argument

Terkadang dalam sebuah program ada argumen yang tidak selalu diisi, misalnya fungsi untuk menghitung diskon saat belanja, jika tidak ada diskon ya tidak menuliskan diskonnya. Nah untuk kasus seperti ini ada yang Namanya optional argument, dimana ada parameter yang memiliki nilai default saat tidak ada argument yang diinput.

Untuk menggunakan optional argument kita dapat langsung mendefinisakan langsung pada fungsi seperti sebagai berikut:

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon) / 100
    return bayar

print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

```
[Running] python -u "d:\Project\Kuliah\PrakAlPro-71251229\Minggu_5\tempCodeRunnerFile.py"
100000.0
90000.0
50000.0
```

Bisa dilihat pada print yang pertama hanya ada uang belanjannya saja tanpa diskon dan tidak error, hal tersebut dapat dilihat pada line 1 saat definisi fungi bahwa argument diskon mempunyai nilai default = 0.

Named Argument

Named argument sangat berguna ketika fungsi memiliki banyak parameter opsional dan kita hanya ingin mengisi beberapa saja. Kita bisa langsung menyebutkan nama parameter mana yang ingin diisi tanpa harus mengikuti urutan.

Normalnya, argumen kita tulis sesuai urutan parameter. Tapi Python punya fitur named argument, di mana kita bisa menyebutkan nama parameter secara langsung saat memanggil fungsi. Kalau pakai cara ini, urutannya tidak harus sama.

Contoh penerapannya:

```
def cetak(a, b, c):
```

```
    print("Nilai a:", a)
    print("Nilai b:", b)
    print("Nilai c:", c)
#cara biasa harus urut
cetak(20, 30, 40)

#bisa acak asal ada argumen = berapa gitu
cetak(b=30, c=40, a=20)
```

```
[Running] python -u "d:\Project\Kuliah\PrakAlPro-71251229\Minggu_5\fungsi.py"
Nilai a: 20
Nilai b: 30
Nilai c: 40
Nilai a: 20
Nilai b: 30
Nilai c: 40
```

Walaupun memiliki cara yang berbeda dalam penulisan argumen tetapi kedua cara tersebut memiliki output yang sama.

Anonymous Function

Lambda function adalah fungsi yang tidak punya nama itulah kenapa disebut "anonymous" (tanpa identitas). Lambda biasanya digunakan untuk fungsi-fungsi sederhana yang hanya perlu satu baris.

Sebagai contoh dalam fungsi tambah 2 bilangan:

Fungsi biasa:

```
def tambah(a, b):
    hasil = a + b
    return hasil

print(tambah(10, 20))
```

Fungsi lambda:

```
tambah = lambda a, b: a + b

print(tambah(10, 20))
```

Fungsi di atas yang ada 3 baris bisa disederhanakan menjadi hanya satu baris menggunakan lambda.

BAGIAN 2: LATIHAN MANDIRI (60%)

Latihan 5.1

Buatlah sebuah fungsi yang dapat menentukan apakah ketiga parameter memenuhi semua ketentuan berikut ini:

• Ketiga parameter tersebut nilainya berbeda semua.

• Ada kemungkinan jika diambil dua parameter dan dijumlahkan hasilnya sama dengan parameter lainnya (yang tersisa). Fungsi tersebut akan menghasilkan nilai True jika semua ketentuan tersebut dipenuhi. Jika tidak terpenuhi maka fungsi akan menghasilkan nilai False. Fungsi anda harus diberi nama cek_angka().

Jawab :

```
def cek_angka(a, b, c):
    semua_beda = (a != b) and (b != c) and (a != c)

    ada_jumlah = (a + b == c) or (a + c == b) or (b + c == a)

    if semua_beda and ada_jumlah:
        hasil = True ,"Semua kondisi terpenuhi"
        return hasil
    elif not semua_beda and not ada_jumlah:
        hasil = False ,"Tidak semua kondisi terpenuhi"
        return hasil
    elif not semua_beda:
        hasil = False ,"Ada angka yang sama"
        return hasil
    else:
        hasil = False ,"Jumlahnya tidak sesuai"
        return hasil

a = int(input("Masukkan bilangan pertama: "))
b = int(input("Masukkan bilangan kedua: "))
c = int(input("Masukkan bilangan ketiga: "))

print(cek_angka(a, b, c))
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_5> python .\latihan5.1.py
Masukkan bilangan pertama: 1
Masukkan bilangan kedua: 2
Masukkan bilangan ketiga: 3
(True, 'Semua kondisi terpenuhi')
```

Penjelasan:

```
     semua_beda = (a != b) and (b != c) and (a != c)
    ada_jumlah = (a + b == c) or (a + c == b) or (b + c == a)
```

Mengecek apakah ketiga angka berbeda satu sama lain menggunakan operator != dan and, dan Ketiga perbandingan harus bernilai True semua agar semua_beda bernilai True. Jika ada satu saja yang sama, hasilnya False.

Pada IF dan ELSE

Akan mengembalikan variabel hasil berisi **tuple** yaitu pasangan nilai (bool, string). Python mengizinkan penulisan True, "pesan" tanpa tanda kurung, yang secara otomatis menjadi tuple.

Lalu ada input dari pengguna yang akan digunakan untuk diikirim ke fungsi cek_angka().

Latihan 5.2

Buat fungsi bernama cek_digit_belakang() yang menentukan apakah minimal dua dari tiga parameter memiliki digit paling kanan yang sama. Fungsi mengembalikan True jika terpenuhi, False jika tidak.

Jawab:

```
def cek_digit_belakang(a, b, c):
    digit_a = a % 10
    digit_b = b % 10
    digit_c = c % 10

    return (digit_a == digit_b) or (digit_b == digit_c) or (digit_a == digit_c)

a = int(input("Masukkan angka pertama : "))
b = int(input("Masukkan angka kedua   : "))
c = int(input("Masukkan angka ketiga  : "))

print(cek_digit_belakang(a, b, c))
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_5> python .\latihan5.2.py
Masukkan angka pertama : 144
Masukkan angka kedua   : 1664
Masukkan angka ketiga  : 12364
True
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_5>
```

Penjelasan:

Digit paling kanan (satuan) dari sebuah bilangan diperoleh menggunakan operator modulo %10.

Ada tiga pasangan kemungkinan yang dicek menggunakan or. Cukup satu pasangan yang sama agar fungsi mengembalikan True.

Latihan 5.3

Buat fungsi-fungsi konversi suhu menggunakan lambda function. Dua fungsi yang harus diimplementasikan:

- Celcius ke Fahrenheit: F = (9/5) × C + 32
- Celcius ke Reamur: R = 0.8 × C

Jawab:

```
celcius_to_fahrenheit = lambda c: (9/5) * c + 32
celcius_to_reamur     = lambda c: 0.8 * c

print(celcius_to_fahrenheit(100))
print(celcius_to_reamur(80))
print(celcius_to_fahrenheit(0))
```

Output:

```
PS D:\Project\Kuliah\PrakAlPro-71251229\Minggu_5> python .\latihan5.3.py
212.0
64.0
32.0
```

Penjelasan:

Setiap lambda terdiri dari tiga bagian: keyword lambda, parameter (bound variable), dan body (ekspresi yang menghasilkan nilai).

Rumus konversi: F = (9/5) × C + 32. Ditulis sebagai lambda dalam variable celcius_to_fahrenheit.

Rumus konversi: R = 0.8 × C. Ditulis sebagai lambda dalam variable celcius_to_reamur

Link Repository Github : https://github.com/Haigry/PrakAlPro-71251229